QUEUELESS – COLLEGE CANTEEN QUEUE MANAGEMENT SYSTEM

CS23333 – Object Oriented Programming using Java and
CS23332 – Database Management Systems project report

Submitted by

JAYASUDHA A – 251001089

KATRINA MARIA ARUL – 251001107

MADHU LEKHA D – 251001136

NIHA ANJUM I – 251001164

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



NOV 2026

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled "QUEUELESS – COLLEGE CANTEEN QUEUE MANAGEMENT SYSTEM" is the bonafide work of "JAYASUDHA A (251001089), KATRINA MARIA ARUL (251001107), MADHU LEKHA D (251001136), NIHA ANJUM I (251001164)" who carried out the project work under my supervision.

SIGNATURE

Dr. P. Neelaveni

HEAD OF THE DEPARTMENT

Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai – 602 105

SIGNATURE

Ms Janani.M

COURSE INCHARGE

Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai – 602 105

This project report is submitted for CS23333 – Object Oriented Programming using Java and CS23332 – Database Management Systems held on
…….

INTERNAL EXAMINER

EXTERNAL EXAMINER

STUDENT DECLARATION

We hereby declare that the project work entitled “QUEUELESS – COLLEGE CANTEEN QUEUE MANAGEMENT SYSTEM” is a record of original work carried out by us for the courses CS23333 – Object Oriented Programming using Java and CS23332 – Database Management Systems, and that all sources used are acknowledged in the references.

Place: Chennai
Date: 05 / 11 / 2026

JAYASUDHA A – 251001089
KATRINA MARIA ARUL – 251001107
MADHU LEKHA D – 251001136
NIHA ANJUM I – 251001164

ACKNOWLEDGEMENT

We thank the management and the Principal of Rajalakshmi Engineering College for the facilities provided for this project. We are grateful to Dr. P. Neelaveni, Head of the Department, and Ms Janani.M, Course Incharge, for their guidance and support. We also thank our faculty members, parents and friends for their help and encouragement.

ABSTRACT

Students in a college canteen do not know how many orders are ahead of them or when their food will be ready, so they keep returning to the counter. QueueLess is a command-line canteen queue management system built with Java, MySQL and JDBC. Students can view the food menu with availability, place an order, receive a token number with an estimated waiting time, and check the order status. Canteen staff can view all orders, mark an order READY or COMPLETED, and mark a food item available or sold out.

The waiting time is calculated by adding the preparation times of the active orders in the queue to the preparation time of the new order, so it becomes lower when earlier orders are ready. JDBC with prepared statements connects the program to the database, and an order is saved using a transaction that is committed or rolled back. The system runs from the Command Prompt.

The report describes the requirements, the design of the program and the five-table database, the important parts of the Java implementation with explanations, the test cases used to verify it, and the limitations and possible future work.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
1	INTRODUCTION	6
2	REQUIREMENTS AND FEASIBILITY	7
3	SYSTEM ANALYSIS AND DESIGN	9
4	DATABASE DESIGN	11
5	IMPLEMENTATION	14
6	VERIFICATION AND TEST PLAN	24
7	SECURITY, LIMITATIONS, AND MAINTENANCE	25
8	CONCLUSION AND FUTURE WORK	25
	REFERENCES	26
	APPENDIX A – SQL SCHEMA	27
	APPENDIX B – OUTPUT SCREENSHOTS	28

1 - INTRODUCTION

1.1 BACKGROUND

Our college has two canteens, Mart and Cafe. A student can place an order and pay using a QR code, but after that the student cannot see how many orders are ahead, how long the food will take or when it is ready. Students keep going back to the counter and wait in the queue unnecessarily. QueueLess models a digital workflow in which a student places an order, gets a token and an estimated waiting time, and checks the order status. It is built as a console prototype so that the logic of tokens, waiting time and order status can be demonstrated and tested before a graphical or mobile version is attempted.

1.2 PROBLEM STATEMENT

To provide a simple, database-backed process in which a student can view the menu, place an order, get a token number with an estimated waiting time and check the order status, while the canteen staff can update the order status and mark food items as sold out.

1.3 OBJECTIVES

- Show the food menu with price and availability (AVAILABLE or SOLD OUT).
- Place an order, generate a token number and show an estimated waiting time.
- Let staff mark an order READY or COMPLETED and update food availability.
- Persist students, food items, orders and order items in MySQL through JDBC.
- Demonstrate Java methods, prepared statements and transaction handling.

1.4 SCOPE

The implemented system is the Java console application in the single file Main.java. It has a student menu, a staff menu and a five-table MySQL database. The waiting-time estimate uses fixed preparation times stored in the database. The system does not include online payment, a website or mobile app, automatic detection of ready food, or staff login.

1.5 TECHNOLOGIES USED

Technology	Purpose
Java (JDK)	Program logic, console menus and order processing
MySQL 8.0	Stores students, menu items, orders and order items
JDBC / MySQL Connector/J	Connects the Java program to the MySQL database
Command Prompt, Notepad	Editing, compiling and running Main.java

1.6 ORGANISATION OF THE REPORT

Chapter 2 lists the requirements and the feasibility of the project. Chapter 3 presents the architecture, data flow, use cases, order-status flow and menu structure. Chapter 4 describes the database. Chapter 5 explains the implementation with code excerpts. Chapter 6 gives the test plan, Chapter 7 covers security, limitations and maintenance, and Chapter 8 concludes with future work. The SQL schema is given in Appendix A.

2 - REQUIREMENTS AND FEASIBILITY

2.1 FUNCTIONAL REQUIREMENTS

ID	Requirement	Where implemented
FR-01	Display the food menu with price and availability.	displayFoodMenu
FR-02	Accept an order only for an available item and a quantity above zero.	placeOrder
FR-03	Find a student by email or create a new student record.	findOrCreateStudent
FR-04	Generate the next token number and save the order and item in one transaction.	getNextToken, placeOrder
FR-05	Calculate the estimated waiting time from the active orders.	getPendingWaitingTime
FR-06	Show the current status of an order using its token number.	checkStatus
FR-07	Let staff view all orders and mark them READY or COMPLETED (COMPLETED only after READY).	viewOrders, updateStatus
FR-08	Let staff mark a food item available or sold out.	updateAvailability
FR-09	Ask again when a menu choice is not a number, and reject an empty student name or email.	readInt, placeOrder

2.2 NON-FUNCTIONAL REQUIREMENTS

- **Usability:** numbered menus, clear prompts and short validation messages in the console.
- **Data integrity:** an order and its item are saved in one transaction, so both succeed or both fail.
- **Security:** prepared statements are used for every user-supplied SQL value.
- **Maintainability:** prices, preparation times and availability are stored in the database, not fixed in the code.

2.3 HARDWARE AND SOFTWARE REQUIREMENTS

Component	Requirement
Hardware and OS	Windows computer, dual-core processor, 4 GB RAM, 1 GB free disk space
Language	Java Development Kit (JDK)
Database	MySQL Server 8.0
Driver and tools	MySQL Connector/J (mysql-connector-j-26.7.0.jar), Command Prompt, Notepad

2.4 FEASIBILITY

Technical: Java, MySQL and the Connector/J driver are sufficient for a small academic application and ran on a normal computer. **Operational:** the menu-driven flow is easy to demonstrate, and staff only update order status and food availability. **Economic:** only free tools and a local MySQL server are used, so no paid hosting is required.

2.5 ASSUMPTIONS AND CONSTRAINTS

- MySQL Server is running on the same computer (localhost, port 3306) and the queueless database is created using Appendix A.
- A student is identified by the email address typed while ordering.
- Each order contains one food item with a quantity; a student places separate orders for different foods.
- Preparation times are fixed values in the FoodMenu table, and the kitchen is assumed to prepare orders one after another.
- Staff update the order status manually; the program cannot detect when food is ready.

3 - SYSTEM ANALYSIS AND DESIGN

3.1 SYSTEM ARCHITECTURE

The student or staff member uses the console menus. Main.java owns the menus and the order logic and sends SQL queries through JDBC to the MySQL database queueless.

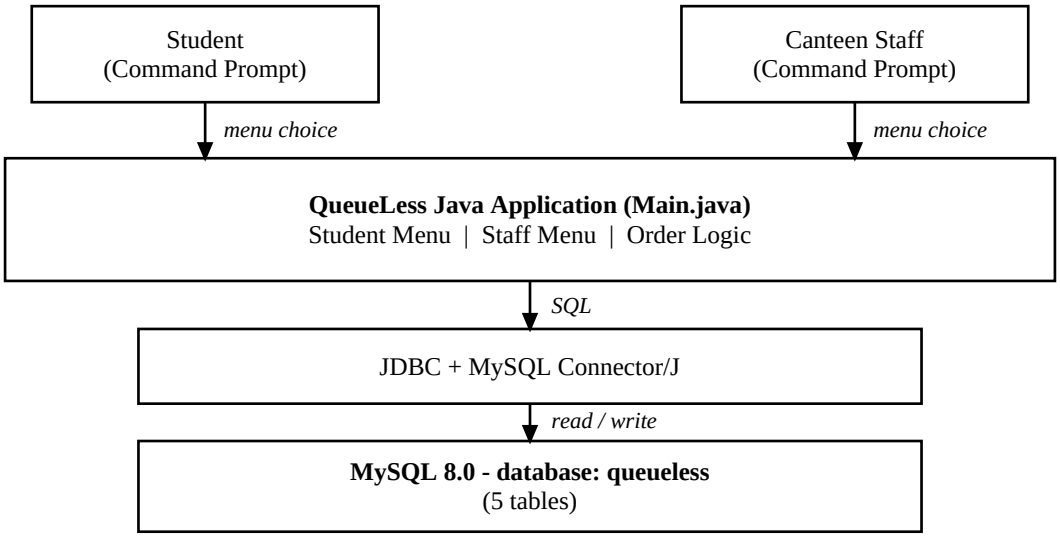


Figure 3.1: Layered organisation of the Java console application

3.2 DATA FLOW DIAGRAM

Students send order and status requests and receive the menu, token, waiting time and status. Staff send status and availability updates and receive the order list.

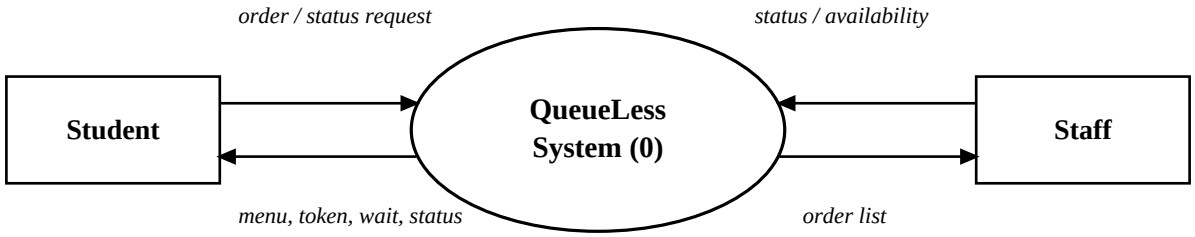


Figure 3.2: Data flow diagram (Level 0)

3.3 USE CASES

Actor	Use cases
Student	View Food Menu; Place Order (token + estimated wait); Check Order Status
Staff	View All Orders; Mark Order READY; Mark Order COMPLETED; Update Food Availability

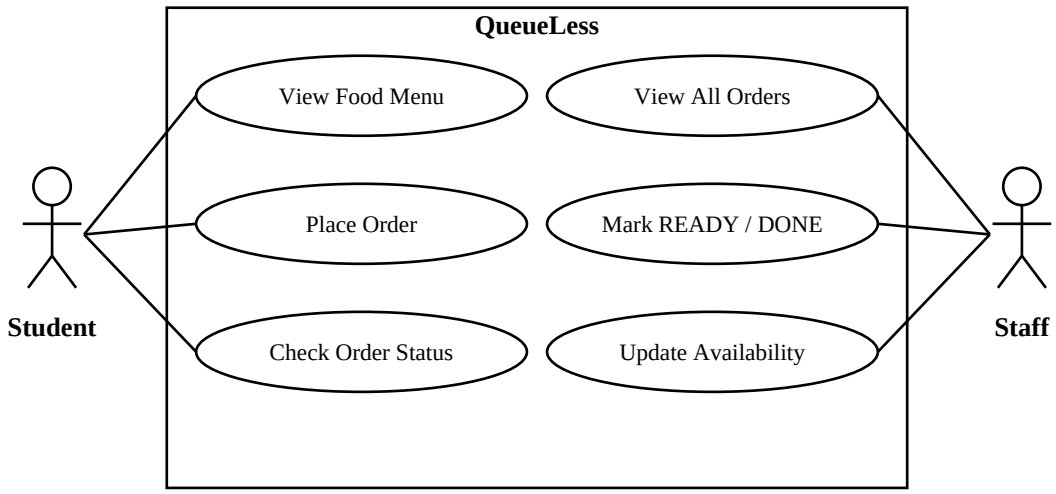


Figure 3.3: Use case diagram

3.4 ORDER STATUS FLOW

An order moves in one direction: NEW, PREPARING, READY and COMPLETED. A new order is stored as NEW. The student sees “Order Received” when the order is placed and PREPARING when the status is checked afterwards. Staff can mark an order READY at any time, and COMPLETED only after READY.

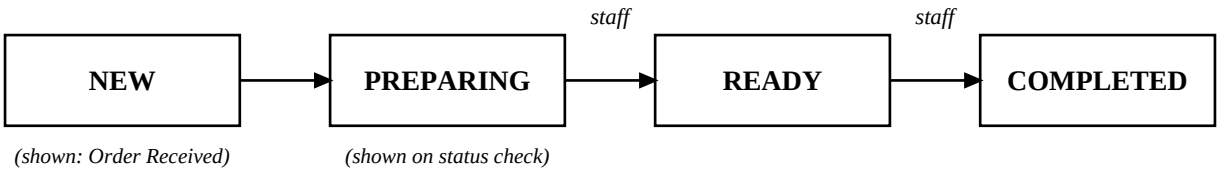


Figure 3.4: Order status flow

3.5 MENU STRUCTURE

The program is driven by three numbered menus. Each menu repeats until the user chooses to go back or exit.

Menu	Options
Main menu	1. Student Menu 2. Staff Menu 3. Exit
Student menu	1. View Food Menu & Availability 2. Place Order 3. Check Order Status 4. Back
Staff menu	1. View All Orders 2. Mark Order as READY 3. Mark Order as COMPLETED 4. Update Food Availability 5. Back

3.6 OBJECT-ORIENTED CONCEPTS USED

Concept	Use in QueueLess
Classes and objects	Main class; Scanner, Connection, Statement and ResultSet objects
Methods (modularity)	Separate methods for menus, orders, status, availability and waiting time
Arrays	food[], price[], prepTime[] and available[] hold the menu in memory
Abstraction	JDBC interfaces hide the MySQL driver details
Exception handling	try–catch for SQLException, ClassNotFoundException and invalid input

4 - DATABASE DESIGN

4.1 TABLES AND COLUMNS USED BY THE PROGRAM

The program uses five MySQL tables in the queueless database. The table below lists the columns that the Java SQL statements read or write. The Staff table is created for future use and is not accessed by the code.

Table	Columns referenced in the Java source	Used by
Students	student_id, student_name, email, phone	findOrCreateStudent
FoodMenu	food_id, food_name, price, prep_time, availability	loadFoodMenu, updateAvailability, viewOrders, checkStatus
Orders	order_id, token_number, student_id, order_status	placeOrder, getNextToken, updateStatus, checkStatus
OrderItems	order_item_id, order_id, food_id, quantity, item_total	placeOrder, getPendingWaitingTime
Staff	staff_id, name, username, password	not used yet

4.2 ENTITY RELATIONSHIPS

Relationship	Type	Linked by
Students – Orders	1 : N	Orders.student_id (FK)
Orders – OrderItems	1 : N	OrderItems.order_id (FK)
FoodMenu – OrderItems	1 : N	OrderItems.food_id (FK)

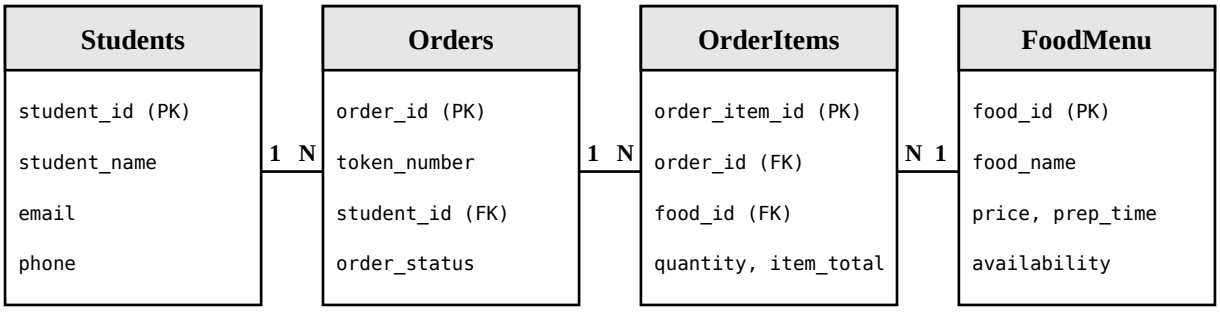


Figure 4.1: Tables and relationships

One student can place many orders, and each order has an order item that points to one food item. Food details are stored once in FoodMenu and student details once in Students, so there is no repeated data.

4.3 DATA DICTIONARY

The data types below are taken from the SQL schema in Appendix A.

Table	Column	Type	Key / constraint
Students	student_id	INT	Primary key, AUTO_INCREMENT
	student_name, email, phone	VARCHAR	email is UNIQUE and NOT NULL
FoodMenu	food_id	INT	Primary key
	food_name, price, prep_time, availability	VARCHAR, DECIMAL, INT, BOOLEAN	availability defaults to TRUE
Orders	order_id	INT	Primary key, AUTO_INCREMENT
	token_number, student_id, order_status, order_date	INT, INT, VARCHAR, TIMESTAMP	student_id is a foreign key; status defaults to 'NEW'
OrderItems	order_item_id	INT	Primary key, AUTO_INCREMENT
	order_id, food_id, quantity, item_total	INT, INT, INT, DECIMAL	order_id and food_id are foreign keys

4.4 DATA INTEGRITY MEASURES IN THE CODE

- The order row and the order item row are inserted inside one transaction with commit and rollback.
- Every SQL value that comes from the user is bound through PreparedStatement placeholders.
- The student is found by email before a new record is created, so the same student is not added twice.
- Primary and foreign keys in the database link the tables (Appendix A).

4.5 SAMPLE DATA

ID	Food item	Price (Rs.)	Prep time
1	Veg Burger	80	3 min
2	Veg Sandwich	50	10 min
3	Margherita Pizza	120	15 min
4	French Fries	60	7 min
5	White Sauce Pasta	100	12 min
6	Veg Momos	70	10 min
7	Paneer Nuggets	90	12 min
8	Veg Shawarma	100	10 min
9	Cheese Maggi	50	5 min
10	Paneer Roll	80	10 min

5 - IMPLEMENTATION

5.1 TECHNOLOGIES AND SOURCE FILES

The application is written in Java and uses MySQL for persistent storage. JDBC (MySQL Connector/J) provides database access, and java.util.Scanner reads the console input. The complete source code is in the single file Main.java; the listings below show the important parts of it.

File	Purpose
Main.java	Student and staff menus, order processing, waiting-time calculation and all JDBC code
mysql-connector-j-26.7.0.jar	MySQL Connector/J driver used to connect Java with MySQL

5.2 JDBC DATABASE CONNECTION

The program asks for the MySQL password at start-up, loads the driver and opens one connection that all methods use. It then resets the AUTO_INCREMENT counters of empty tables and loads the food menu.

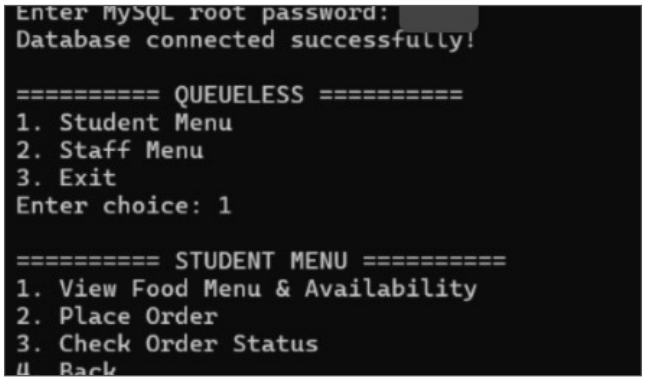


Figure 5.1: Database connection and main menu

JAVA CODE – Connecting to MySQL (excerpt of Main.main):

```
System.out.print("Enter MySQL root password: ");
String password = sc.nextLine();

Class.forName("com.mysql.cj.jdbc.Driver");
con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/queueless", "root", password);
System.out.println("\nDatabase connected successfully!");

resetAutoIncrementIfEmpty("OrderItems");
resetAutoIncrementIfEmpty("Orders");
resetAutoIncrementIfEmpty("Students");
loadFoodMenu();
```

Code	Explanation
`sc.nextLine()`	Reads the MySQL password typed by the user, so it is not written in the source code.
`jdbc:mysql://localhost:3306/queueless`	JDBC address: MySQL on localhost, port 3306, database queueless.
`DriverManager.getConnection(...)`	Opens and returns the Connection; it throws SQLException if the server or password is wrong.

Code	Explanation
`resetAutoIncrementIfEmpty(...)`	Counts the rows of a table and, if it is empty, sets AUTO_INCREMENT back to 1 so fresh test orders start from 1.

5.3 FOOD MENU

loadFoodMenu() reads every row of the FoodMenu table into the arrays food[], price[], prepTime[] and available[]. displayFoodMenu() loads the menu again each time it is shown, so a change made by the staff appears to the students at once.

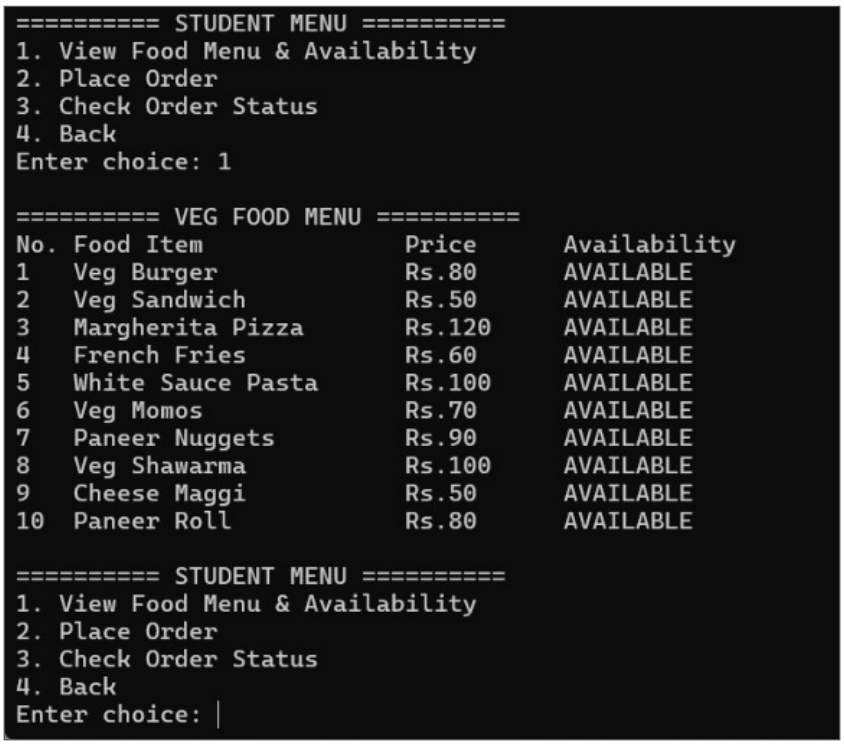


Figure 5.2: Food menu with availability

JAVA CODE – Loading and displaying the menu:

```
String sql = "SELECT food_id, food_name, price, prep_time, availability "
           + "FROM FoodMenu";
try (Statement st = con.createStatement();
     ResultSet rs = st.executeQuery(sql)) {
    while (rs.next()) {
        int id = rs.getInt("food_id");
        food[id - 1]      = rs.getString("food_name");
        price[id - 1]     = rs.getBigDecimal("price").intValue();
        prepTime[id - 1]  = rs.getInt("prep_time");
        available[id - 1] = rs.getBoolean("availability");
    }
}
// in displayFoodMenu()
System.out.printf("%-4d %-23s Rs.%-9d %-15s%n", i + 1, food[i], price[i],
        available[i] ? "AVAILABLE" : "SOLD OUT");
```

Code	Explanation
`while (rs.next())`	Moves through the rows returned by the query, one food item at a time.
`food[id - 1] = ...`	Stores each column in the array position of that food id (food id 1 is index 0).
`rs.getBoolean("availability")`	Reads the availability flag that the staff can change.
`available[i] ? "AVAILABLE" : "SOLD OUT"`	Prints the availability text beside each food item in the menu.

5.4 PLACING AN ORDER

placeOrder() shows the menu, reads the food number and quantity, and rejects a sold-out item or a quantity of zero or less. The student name, email and phone are then read, and the name and email cannot be empty.

```
========== VEG FOOD MENU ==========
No. Food Item              Price      Availability
1   Veg Burger             Rs.80      AVAILABLE
2   Veg Sandwich           Rs.50      AVAILABLE
3   Margherita Pizza       Rs.120     AVAILABLE
4   French Fries           Rs.60      AVAILABLE
5   White Sauce Pasta      Rs.100     AVAILABLE
6   Veg Momos              Rs.70      AVAILABLE
7   Paneer Nuggets         Rs.90      AVAILABLE
8   Veg Shawarma           Rs.100     AVAILABLE
9   Cheese Maggi           Rs.50      AVAILABLE
10  Paneer Roll            Rs.80      AVAILABLE

Select food item (1-10): 2
Enter quantity: 2
Enter student name: Jayasudha A
Enter student email: jayasudhaa@gmail.com
Enter student phone: 9876543907

========== ORDER CONFIRMATION ==========
Order placed successfully!
Token Number: 102
Food: Veg Sandwich
Quantity: 2
Total Amount: Rs.100
Estimated Waiting Time: 10 minutes
Order Status: NEW
```

Figure 5.3: Order confirmation with token and estimated waiting time

JAVA CODE – Validating the food item and quantity (excerpt of placeOrder):

```
int index = selected - 1;
if (!available[index]) {
    System.out.println(food[index] + " is SOLD OUT!");
    return;
}
System.out.print("Enter quantity: ");
int qty = readInt();
if (qty <= 0) {
    System.out.println("Quantity must be greater than zero!");
    return;
}
```

Code	Explanation
`!available[index]`	True when the staff have marked the item SOLD OUT; the order is stopped before anything is saved.
`readInt()`	Keeps asking until the user types a valid number, so a letter cannot crash the program.
`qty <= 0`	Rejects a quantity that is zero or negative.

Next, the student is found by email or created, and the next token number is generated from the highest token already stored (the first token is 101).

JAVA CODE – Student, token and waiting time (excerpt of placeOrder and its helper methods):

```
int studentId = findOrCreateStudent(name, email, phone);
int token = getNextToken();
int waitingTime = getPendingWaitingTime() + prepTime[index] * qty;

// findOrCreateStudent
String findSql = "SELECT student_id FROM Students WHERE email = ?";

// getNextToken
String sql = "SELECT COALESCE(MAX(token_number), 100) + 1 "
           + "AS next_token FROM Orders";
```

Code	Explanation
`WHERE email = ?`	Looks for an existing student with the typed email; a new row is inserted only if none is found.
`COALESCE(MAX(token_number), 100) + 1`	Gives 101 for the first order, and one more than the highest token after that.
`getPendingWaitingTime() + prepTime[index] * qty`	Time of the active orders plus the preparation time of this order (explained in 5.5).

The order and its item are saved together inside one transaction, which is committed only if both inserts succeed.

JAVA CODE – Saving the order in a transaction (excerpt of placeOrder):

```
con.setAutoCommit(false);
try {
    String orderSql = "INSERT INTO Orders (token_number, student_id, order_status) "
                    + "VALUES (?, ?, 'NEW')";
    try (PreparedStatement ps = con.prepareStatement(orderSql,
            Statement.RETURN_GENERATED_KEYS)) {
        ps.setInt(1, token);
        ps.setInt(2, studentId);
        ps.executeUpdate();
        // orderId is read with ps.getGeneratedKeys()
    }
    String itemSql = "INSERT INTO OrderItems "
        + "(order_id, food_id, quantity, item_total) VALUES (?, ?, ?, ?)";
    try (PreparedStatement ps = con.prepareStatement(itemSql)) {
        ps.setInt(1, orderId);
        ps.setInt(2, selected);
        ps.setInt(3, qty);
        ps.setBigDecimal(4, new BigDecimal(price[index] * qty));
        ps.executeUpdate();
    }
    con.commit();
} catch (SQLException e) {
    con.rollback();
    throw e;
} finally {
    con.setAutoCommit(true);
}
```

Code	Explanation
`con.setAutoCommit(false)`	Starts a manual transaction: nothing is permanent until commit().
`VALUES (?, ?, 'NEW')`	A new order is always stored with the status NEW.
`Statement.RETURN_GENERATED_KEYS`	Asks MySQL to return the auto-generated order_id, which is used for the order item.
`ps.setInt(...) / ps.setBigDecimal(...)`	Binds values to the ? placeholders, so user text is never joined into the SQL string.
`con.commit() / con.rollback()`	Both rows are saved together; if anything fails both are undone and the error is printed.
`finally { con.setAutoCommit(true); }`	Restores normal auto-commit mode.

5.5 ESTIMATED WAITING TIME

The waiting time is the total preparation time of all active orders (status NEW or PREPARING) in the queue, plus the preparation time of the new order. READY and COMPLETED orders are not counted, so the estimate for the next order becomes lower when earlier orders are ready.

Waiting time = Σ (prep_time × quantity) of all NEW / PREPARING items + (prep_time × quantity) of the new order

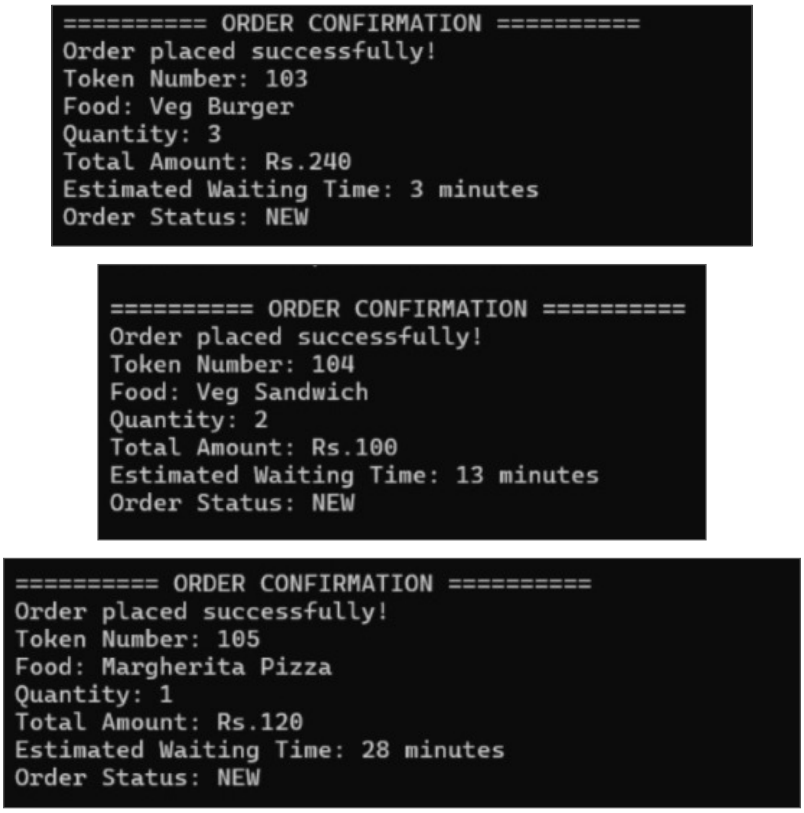


Figure 5.4: Estimated waiting time for three orders

JAVA CODE – getPendingWaitingTime:

```
String sql = "SELECT oi.food_id, oi.quantity "
           + "FROM Orders o "
           + "JOIN OrderItems oi ON o.order_id = oi.order_id "
           + "WHERE o.order_status IN ('NEW', 'PREPARING')";
int total = 0;
while (rs.next()) {
    int foodId = rs.getInt("food_id");
    int qty = rs.getInt("quantity");
    total += prepTime[foodId - 1] * qty;
}
return total;
```

Code	Explanation
`o.order_status IN ('NEW', 'PREPARING')`	Selects only the active orders; READY and COMPLETED orders are left out.
`JOIN OrderItems oi ...`	Joins each order with its food id and quantity.
`total += prepTime[foodId - 1] * qty`	Adds the preparation time of each active order (time per item x quantity).

Example: a Veg Burger (3 min), Veg Sandwich (10 min) and Cheese Maggi (5 min) are ordered one after another.

Step	Active orders	Estimate for the new order
Order 1: Veg Burger	none	3 min
Order 2: Veg Sandwich	Order 1	3 + 10 = 13 min
Order 3: Cheese Maggi	Orders 1, 2	3 + 10 + 5 = 18 min
Order 1 marked READY, new Cheese Maggi order	Orders 2, 3	10 + 5 + 5 = 20 min (was 23 min)

5.6 VIEWING ALL ORDERS

Staff choose option 1 in the staff menu to see every order. viewOrders() joins the Orders, OrderItems and FoodMenu tables and prints one row per order item with the order ID, token, food name, quantity and status. A stored status of NEW is displayed as PREPARING, and the total number of rows is printed at the end.

JAVA CODE – viewOrders (excerpt):

```
String sql = "SELECT o.order_id, o.token_number, o.order_status, "
           + "fm.food_name, oi.quantity "
           + "FROM Orders o "
           + "JOIN OrderItems oi ON o.order_id = oi.order_id "

           + "JOIN FoodMenu fm ON oi.food_id = fm.food_id "
           + "ORDER BY o.order_id";
...
String status = rs.getString("order_status");
if (status.equals("NEW")) {
    status = "PREPARING";
}
```

Code	Explanation
`JOIN FoodMenu fm ON oi.food_id = fm.food_id`	Replaces the food id by the food name so the staff can read the list easily.
`ORDER BY o.order_id`	Lists the orders in the order in which they were placed.
`status = "PREPARING"`	Shows NEW orders to the staff as PREPARING, the same word the student sees.

5.7 UPDATING THE ORDER STATUS

The staff enter the token number and choose READY or COMPLETED. updateStatus() first reads the current status and applies the rules before the UPDATE statement is run.

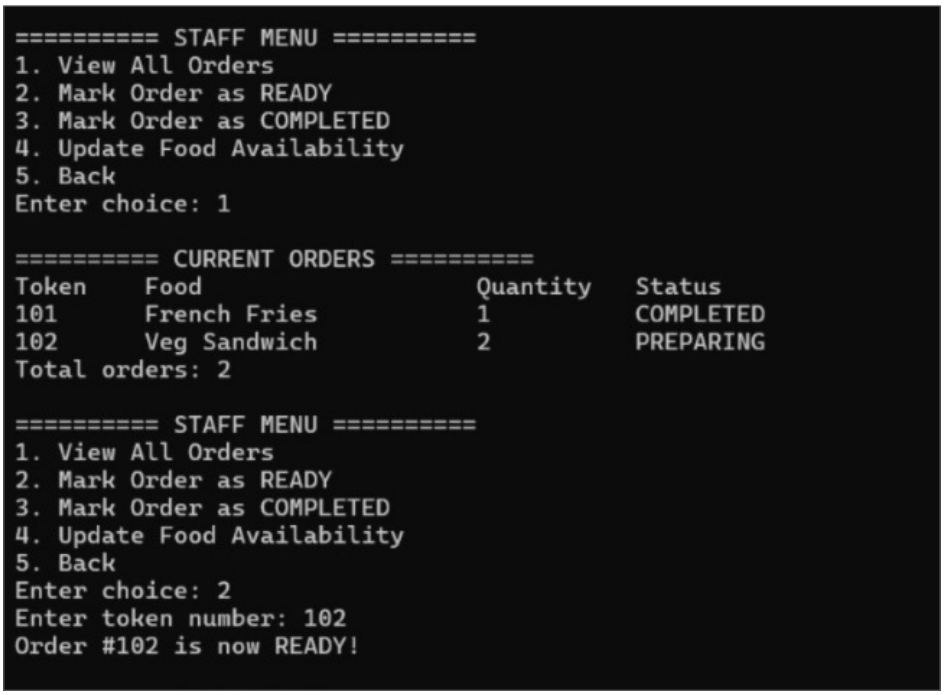


Figure 5.5: Staff view of all orders and marking an order READY

JAVA CODE – updateStatus (excerpt):

```
if (newStatus.equals("READY")) {
    if (oldStatus.equals("READY")) {
        System.out.println("Order is already READY!");
        return;
    }
    if (oldStatus.equals("COMPLETED")) {
        System.out.println("Order is already COMPLETED!");
        return;
    }
}
if (newStatus.equals("COMPLETED") && !oldStatus.equals("READY")) {
    System.out.println("Mark the order READY first!");
    return;
}
String updateSql = "UPDATE Orders SET order_status = ? WHERE token_number = ?";
```

Code	Explanation
`oldStatus.equals("READY")`	Stops the same order from being marked READY twice.
`!oldStatus.equals("READY")`	An order can be COMPLETED only after it is READY.
`UPDATE Orders SET order_status = ? ...`	Saves the new status for the token; the order then drops out of the waiting-time calculation.

5.8 UPDATING FOOD AVAILABILITY

The staff select a food item and mark it AVAILABLE or SOLD OUT. The change is saved in the FoodMenu table, so the Java code need not be edited.

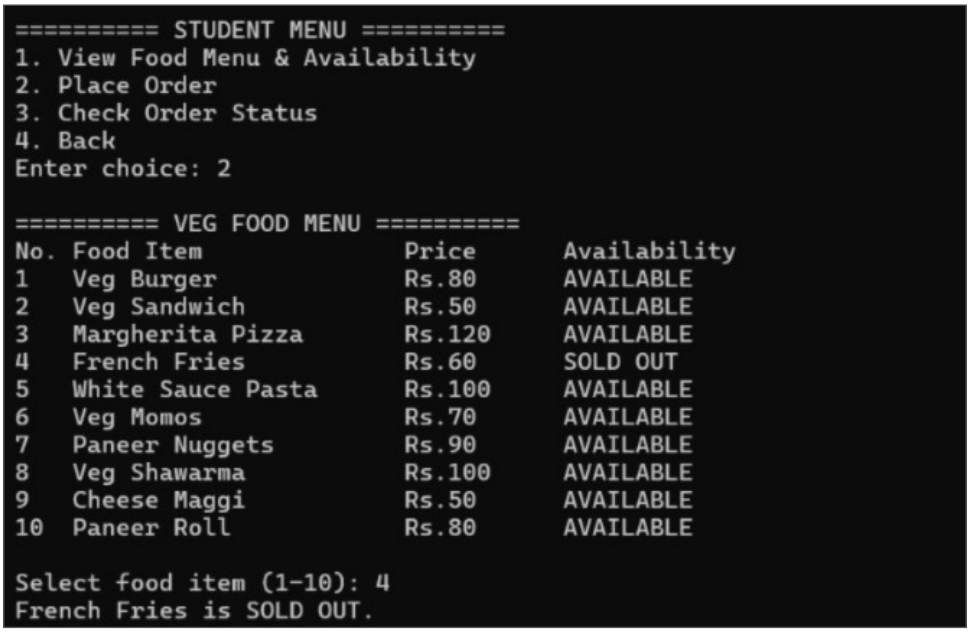


Figure 5.6: Food item marked SOLD OUT and blocked for students

JAVA CODE – updateAvailability (excerpt):

```
boolean newAvailability = choice == 1;
String sql = "UPDATE FoodMenu SET availability = ? WHERE food_id = ?";
try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setBoolean(1, newAvailability);
    ps.setInt(2, selected);
    int changed = ps.executeUpdate();
    if (changed == 0) {
        System.out.println("Food item not found in database!");
        return;
    }
    available[selected - 1] = newAvailability;
}
```

Code	Explanation
`choice == 1`	Option 1 means AVAILABLE and option 2 means SOLD OUT.
`ps.executeUpdate()`	Returns the number of rows changed; 0 means the food id does not exist.
`available[selected - 1] = ...`	Updates the array so the next menu display shows the new availability.

5.9 CHECKING THE ORDER STATUS

The student enters the token number. checkStatus() joins the Orders, OrderItems and FoodMenu tables and prints the food, quantity and a status message.

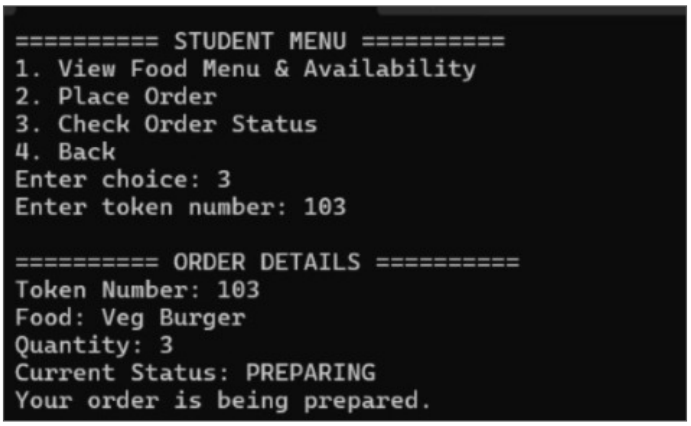


Figure 5.7: Order status for a token

JAVA CODE – checkStatus (excerpt):

```
String sql = "SELECT o.token_number, o.order_status, fm.food_name, oi.quantity "
           + "FROM Orders o "
           + "JOIN OrderItems oi ON o.order_id = oi.order_id "
           + "JOIN FoodMenu fm ON oi.food_id = fm.food_id "
           + "WHERE o.token_number = ?";
...

if (status.equals("NEW") || status.equals("PREPARING")) {
    System.out.println("Current Status: PREPARING");
    System.out.println("Your order is being prepared.");
} else if (status.equals("READY")) {
    System.out.println("Current Status: READY");
    System.out.println("Please collect your order.");
} else if (status.equals("COMPLETED")) {
    System.out.println("Current Status: COMPLETED");
    System.out.println("Order collected. Thank you!");
}
```

Code	Explanation
`WHERE o.token_number = ?`	Finds the order of the typed token; “Token number not found!” is printed if there is none.
`status.equals("NEW") || ...`	A NEW order is shown to the student as PREPARING.
`READY / COMPLETED messages`	Tell the student to collect the order, or that it has been collected.

5.10 COMPILING AND RUNNING

Open Command Prompt, go to the project folder, then compile and run Main.java with the MySQL Connector/J jar in the classpath.

JAVA CODE – Commands:

```
cd "C:\Users\Sakthi Rekha\QueueLess"
javac -cp ".;C:\Users\Sakthi Rekha\Downloads\QueueLess\mysql-connector-j-26.7.0.jar" Main.java
java -cp ".;C:\Users\Sakthi Rekha\Downloads\QueueLess\mysql-connector-j-26.7.0.jar" Main
```

6 - VERIFICATION AND TEST PLAN

6.1 TEST PLAN

The table lists the test cases designed from the behaviour of the code, with the figure that shows the result where a screenshot is included.

Test	Input / condition	Expected result	Evidence
TC-01	View the food menu	Ten items with price and AVAILABLE / SOLD OUT	Figure 5.2
TC-02	Place an order for an available item	Token, total amount, estimated wait and “Order Received”	Figure 5.3
TC-03	Place an order for a SOLD OUT item	“<food> is SOLD OUT!”; no order saved	Figure 5.6
TC-04	Quantity 0, or empty name / email	Error message; order is not placed	Manual run
TC-05	Place three orders (Burger, Sandwich, Maggi)	Estimates 3, 13 and 18 minutes	Figure 5.4
TC-06	Mark the first order READY, then place a new order	Estimate is lower by the READY order’s time	Figure 5.5
TC-07	Check status with a valid token	Details and status message	Manual run

Test	Input / condition	Expected result	Evidence
TC-08	Check status with an unknown token	“Token number not found!”	Manual run
TC-09	Mark COMPLETED before READY	“Mark the order READY first!”	Manual run
TC-10	Type a letter in a menu	“Enter a valid number:” and asks again	Manual run
TC-11	Order again using the same email	The existing student ID is reused; no duplicate student	Manual run
TC-12	Staff chooses View All Orders	Every order item listed with token, food, quantity and status	Figure 5.5

6.2 TEST RESULT

All the test cases gave the expected results when run with the MySQL database.
Old test orders were removed with DELETE so that fresh orders could be tested.

7 - SECURITY, LIMITATIONS, AND MAINTENANCE

7.1 SECURITY MEASURES PRESENT IN THE CODE

- All SQL that uses variable values goes through prepared statements, which reduces the SQL-injection risk.
- The MySQL password is typed at run time and is not stored in the source code.
- An order and its item are saved in one transaction; if anything fails, it is rolled back.
- Menu input is read with readInt(), so a wrong type of input does not crash the program.

7.2 LIMITATIONS

- The staff menu has no login; the Staff table exists but is not used by the code yet.
- Preparation times are fixed and staff update the status manually; the system cannot detect when food is ready.
- The estimate is calculated when the order is placed and does not count the time already spent on preparation.
- One food item is stored per order, and the estimate assumes orders are made one after another.
- The next token is the highest stored token plus one, so two terminals ordering at the same moment could receive the same token.
- It is a command-line application, with no online payment, website or mobile app.

7.3 MAINTENANCE

- Change prices, preparation times and availability in the FoodMenu table without editing the Java code.
- Take regular backups of the queueless database.
- Clear old orders with DELETE; the program resets AUTO_INCREMENT when a table is empty.
- Update the jar name in the compile and run commands when the Connector/J version changes.

8 - CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

QueueLess shows students the status of their order and an estimated waiting time, so they need not keep going to the counter. It uses Java, JDBC and a five-table MySQL database, and demonstrates methods, exception handling, transactions and the INSERT, SELECT, UPDATE and DELETE operations. The console screenshots confirm the menu, order placement and status update.

8.2 FUTURE ENHANCEMENTS

- Staff login using the Staff table.
- A web or mobile application with a live status screen.
- Several food items in one order and a live countdown timer.
- Online payment and a notification when the order is READY.
- Support for both canteens, Mart and Cafe.

REFERENCES

- Oracle. *Java Documentation.* https://docs.oracle.com/en/java/
- Oracle. *JDBC Basics.* https://docs.oracle.com/javase/tutorial/jdbc/
- MySQL. *MySQL 8.0 Reference Manual.* https://dev.mysql.com/doc/refman/8.0/en/
- MySQL. *Connector/J Developer Guide.* https://dev.mysql.com/doc/connector-j/en/

APPENDIX A – SQL SCHEMA

The schema below is the relational design, written to match the columns that the Java code uses. Match the column sizes and constraints to your local MySQL database.

```
CREATE DATABASE queueless;
USE queueless;

CREATE TABLE Students (
    student_id   INT AUTO_INCREMENT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    email        VARCHAR(100) NOT NULL UNIQUE,
    phone        VARCHAR(15)
);

CREATE TABLE FoodMenu (
    food_id      INT PRIMARY KEY,
    food_name    VARCHAR(100) NOT NULL,
    price        DECIMAL(8,2) NOT NULL,
    prep_time    INT NOT NULL,
    availability BOOLEAN DEFAULT TRUE
);

CREATE TABLE Orders (
    order_id     INT AUTO_INCREMENT PRIMARY KEY,
    token_number INT NOT NULL,
    student_id   INT NOT NULL,
    order_status VARCHAR(20) DEFAULT 'NEW',
    order_date   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
);

CREATE TABLE OrderItems (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id      INT NOT NULL,
    food_id       INT NOT NULL,
    quantity      INT NOT NULL,
    item_total    DECIMAL(8,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (food_id)  REFERENCES FoodMenu(food_id)
);

CREATE TABLE Staff (
    staff_id INT AUTO_INCREMENT PRIMARY KEY,
    name     VARCHAR(100) NOT NULL,
    username VARCHAR(50)  NOT NULL UNIQUE,
    password VARCHAR(100) NOT NULL
);

INSERT INTO FoodMenu VALUES
(1,'Veg Burger',80,3,TRUE),       (2,'Veg Sandwich',50,10,TRUE),
(3,'Margherita Pizza',120,15,TRUE),(4,'French Fries',60,7,TRUE),
(5,'White Sauce Pasta',100,12,TRUE),(6,'Veg Momos',70,10,TRUE),
(7,'Paneer Nuggets',90,12,TRUE),  (8,'Veg Shawarma',100,10,TRUE),
(9,'Cheese Maggi',50,5,TRUE),     (10,'Paneer Roll',80,10,TRUE);
```

Appendix B – Output Screenshots

All seven supplied screenshots appear with captions and explanations in the sections named below. The table summarises what each one shows; all of them come from the implemented Java console application.

Figure	Screenshot content	Source	Where
Fig. 5.1	Database connected message and the main menu with the Student Menu opened	Java console application	Section 5.2
Fig. 5.2	Student menu option 1: veg food menu with price and availability (all items AVAILABLE)	Java console application	Section 5.3
Fig. 5.3	Order placed: token 102, Veg Sandwich × 2, total Rs.100, estimated waiting time 10 minutes	Java console application	Section 5.4
Fig. 5.4	Three orders (tokens 103, 104, 105) with estimated waiting times of 3, 13 and 28 minutes	Java console application	Section 5.5
Fig. 5.5	Staff menu: all orders listed, then order #102 marked READY	Java console application	Section 5.7
Fig. 5.6	French Fries marked SOLD OUT and blocked when a student selects it	Java console application	Section 5.8
Fig. 5.7	Check order status: token 103, Veg Burger, status PREPARING	Java console application	Section 5.9